

WEEK-2 MODULE

1. > Fraud Transaction Detection (Linear Search)

Statement: A bank stores transaction ID's in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

Solution! Logic

- (i) Linear search checks each element one by one.
- (ii) Since transaction ID's are not sorted, Binary search can't be used.
- (iii) Traverse the entire array and compare each ID with the suspicious transaction ID.
- (iv) If found, print "Fraud Transaction found".
- (v) Otherwise print "Transaction not found".

Code

```
import java.util.Scanner;  
public class FraudTransactionDetection {  
    public main static void main (String[] args) {  
        try (Scanner sc = new Scanner (System.in)) {  
            System.out.println("Enter no. of transactions: ");  
            int n = sc.nextInt();  
            int[] transactions = new int[n];  
            System.out.println("Enter transactions: ");
```

```

for (int i = 0; i < n; i++) {
    transactions[i] = sc.nextInt();
}

System.out.println("Enter target transaction: ");
int target = sc.nextInt();

boolean found = false;
for (int i = 0; i < n; i++) {
    if (transactions[i] == target) {
        found = true;
        break;
    }
}

if (found)
    System.out.println("Fraud Transaction Found");
else
    System.out.println("Transaction not Found");
}
}

```

Time Complexity : $O(N)$

Space Complexity : $O(1)$

2. > Server log search (Binary Search)

Statement : A cloud company stores server log ID's in sorted order. Engineers need to quickly verify if a log ID exists.

Solution : Logic

- i) Binary search works only on sorted arrays.
- ii) Compare the middle element with the target log ID.
- iii) If equals, log is found.

J FraudTransactionDetection.java > FraudTransactionDetection > main(String[] args)

```
1 import java.util.Scanner;
2
3 public class FraudTransactionDetection {
4     Run main | Debug main
5     public static void main(String[] args) {
6
7         try( Scanner sc = new Scanner(System.in)){
8
9             System.out.println("Enter number of transactions:");
10            int n = sc.nextInt();
11
12            int[] transactions = new int[n];
13
14            System.out.println("Enter transactions:");
15            for(int i = 0; i < n; i++) {
16                transactions[i] = sc.nextInt();
17            }
18
19            System.out.println("Enter target transaction:");
20            int target = sc.nextInt();
21
22            boolean found = false;
23
24            for(int i = 0; i < n; i++) {
25                if(transactions[i] == target) {
26                    found = true;
27                    break;
28                }
29            }
30
31            if(found)
32                System.out.println("Fraud Transaction Found");
33            else
34                System.out.println("Transaction Not Found");
35        }
36    }
37 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Harshit Puri\DSA Java> cd "c:\Users\Harshit Puri\DSA Java\" ; if ($?) { javac FraudTransactionDetection.java } ; if ($?) { java FraudTransactionDetection }
Enter number of transactions:
3
Enter transactions:
15
12
26
Enter target transaction:
26
Fraud Transaction Found
PS C:\Users\Harshit Puri\DSA Java> cd "c:\Users\Harshit Puri\DSA Java\" ; if ($?) { javac FraudTransactionDetection.java } ; if ($?) { java FraudTransactionDetection }
Enter number of transactions:
3
Enter transactions:
15
66
321
Enter target transaction:
20
Transaction Not Found
PS C:\Users\Harshit Puri\DSA Java> |
```

- (iv) If target is smaller, search the left half, otherwise search the right half.
- (v) Repeat until the element is found or search space becomes empty.

Code

```
import java.util. Scanner;
public class ServerLogSearch {
    public static void main (String [] args) {
        try (Scanner sc = new Scanner (System.in)) {
            System.out.println ("Enter no. of ID's: ");
            int m = sc.nextInt ();
            int [] logs = new int [m];

            System.out.println ("Enter ID's: ");
            for (int i = 0; i < m; i++) {
                logs [i] = sc.nextInt ();
            }

            System.out.println ("Enter Target ID: ");
            int target = sc.nextInt ();

            int low = 0;
            int high = m - 1;
            boolean found = false;
            while (low <= high) {
                int mid = low + (high - low) / 2;
                if (logs [mid] == target) {
                    found = true;
                    break;
                }
            }
        }
    }
}
```

```

    else if (logs[mid] < target) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}
if (found)
    System.out.println("Log Found");
else
    System.out.println("Log Not Found");
}
}
}

```

Time Complexity: $O(\log N)$

Space Complexity: $O(1)$

3. > Sort Employee Salaries (Bubble Sort)

Problem: A company wants to sort employees salaries in ascending order for payroll analysis.

Solution: Logic

- (i) Bubble sort repeatedly compares adjacent elements.
- (ii) If the current salary is greater than the next salary, swap them.
- (iii) After each pass, the largest element moves to its correct position.
- (iv) Continue until all passes are completed.
- (v) The array becomes sorted in ascending order.

J ServerLogSearch.java >  ServerLogSearch >  main(String[] args)

```
1  import java.util.Scanner;
2
3  public class ServerLogSearch {
    Run main | Debug main
4      public static void main(String[] args) {
5          try(Scanner sc = new Scanner(System.in)){
6
7              System.out.println("Enter number of ID's:");
8              int n = sc.nextInt();
9
10             int[] logs = new int[n];
11
12             System.out.println("Enter ID's:");
13             for(int i = 0; i < n; i++) {
14                 logs[i] = sc.nextInt();
15             }
16
17             System.out.println("Enter target ID:");
18             int target = sc.nextInt();
19
20             int low = 0;
21             int high = n - 1;
22             boolean found = false;
23
24             while(low <= high) {
25                 int mid = low + (high - low) / 2;
26
27                 if(logs[mid] == target) {
28                     found = true;
29                     break;
30                 }
31                 else if(logs[mid] < target) {
32                     low = mid + 1;
33                 }
34                 else {
35                     high = mid - 1;
36                 }
37             }
38
39             if(found)
40                 System.out.println("Log Found");
41             else
42                 System.out.println("Log Not Found");
43         }
44     }
45 }
46 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Harshit Puri\DSA Java> cd "c:\Users\Harshit Puri\DSA Java\" ; if (\$?) { javac ServerLogSearch.java } ; if (\$?) { java ServerLogSearch }

Enter number of ID's:

3

Enter ID's:

15

32

65

Enter target ID:

65

Log Found

PS C:\Users\Harshit Puri\DSA Java> cd "c:\Users\Harshit Puri\DSA Java\" ; if (\$?) { javac ServerLogSearch.java } ; if (\$?) { java ServerLogSearch }

Enter number of ID's:

3

Enter ID's:

15

98

44

Enter target ID:

3

Log Not Found

PS C:\Users\Harshit Puri\DSA Java> |

Code

```
import java.util.Scanner;
public class EmployeeSalarySort {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            System.out.println("Enter no. of employees: ");
            int n = sc.nextInt();
            System.out.println("Enter salaries of employees: ");
            int[] salary = new int[n];
            for (int i = 0; i < n; i++) {
                salary[i] = sc.nextInt();
            }
            for (int i = 0; i < n - 1; i++) {
                for (int j = 0; j < n - i - 1; j++) {
                    if (salary[j] > salary[j + 1]) {
                        int temp = salary[j];
                        salary[j] = salary[j + 1];
                        salary[j + 1] = temp;
                    }
                }
            }
            System.out.println("Salaries in sorted order: ");
            for (int i = 0; i < n; i++) {
                System.out.print(salary[i] + " ");
            }
        }
    }
}
```

Time Complexity :- $O(N^2)$

Space Complexity :- $O(1)$

EmployeeSalarySort.java > EmployeeSalarySort > main(String[] args)

```
1  import java.util.Scanner;
2
3  public class EmployeeSalarySort {
    Run main | Debug main
4      public static void main(String[] args) {
5          try( Scanner sc = new Scanner(System.in)){
6
7              System.out.println("Enter Number of Employees: ");
8              int n = sc.nextInt();
9
10             System.out.println("Enter Salaries of Employees: ");
11             int[] salary = new int[n];
12
13             for(int i = 0; i < n; i++) {
14                 salary[i] = sc.nextInt();
15             }
16
17             for(int i = 0; i < n - 1; i++) {
18                 for(int j = 0; j < n - i - 1; j++) {
19
20                     if(salary[j] > salary[j + 1]) {
21                         int temp = salary[j];
22                         salary[j] = salary[j + 1];
23                         salary[j + 1] = temp;
24                     }
25                 }
26             }
27
28             System.out.println("Salaries in Sorted Order: ");
29             for(int i = 0; i < n; i++) {
30                 System.out.print(salary[i] + " ");
31             }
32         }
33     }
34 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Harshit Puri\DSA Java> cd "c:\Users\Harshit Puri\DSA Java\" ; if ($?) { javac EmployeeSalarySort.java } ; if ($?) { java EmployeeSalarySort }
Enter Number of Employees:
3
Enter Salaries of Employees:
12
6
78
Salaries in Sorted Order:
6 12 78
PS C:\Users\Harshit Puri\DSA Java> |
```

4. > Sort Website Response Time (Insertion Sort)

Problem: A web monitoring system records website response times. New data comes continuously and must be inserted into correct position.

Solution:- Logic

- (i) Insertion sort assumes the first element is already sorted.
- (ii) Pick the next element and place it in its correct position.
- (iii) shift larger element one position to the right.
- (iv) Insert the selected element at the correct location.
- (v) Repeat for all elements until the array becomes sorted.

Code

```
import java.util.Scanner;

public class WebsiteResponseSort {

    public static void main (String [] args) {

        try (Scanner sc = new Scanner (System.in)) {

            System.out.println ("Enter no. of Responses: ");
            int n = sc.nextInt();
            System.out.println ("Enter Responses: ");
            int [] response = new int [n];
            for (int i = 0; i < n; i++) {
                response [i] = sc.nextInt();
            }
            for (int i = 1; i < n; i++) {
                int key = response [i];
                int j = i - 1;
                while (j >= 0 && response [j] > key) {
                    response [j+1] = response [j];
                    j--;
                }
            }
        }
    }
}
```

```
response[j+1] = key;
```

```
}
```

```
system.out.println("Responses in sorted order: ");
```

```
for (int i=0; i<n; i++) {
```

```
    system.out.print(response[i] + " ");
```

```
}
```

```
}
```

Time Complexity: $O(N^2)$

Space Complexity: $O(1)$

5. > Priority Bug Reports (Selection Sort)

Problem: A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order.

Solution: Logic

- (i) Selection sort finds the minimum elements from the unsorted part.
- (ii) Swap it with the first unsorted position.
- (iii) After every pass, one element reaches its correct position.
- (iv) Continue selecting the smallest remaining element.
- (v) Eventually the array becomes sorted in ascending order.

Code

```
import java.util.Scanner;
```

```
public class BugReportPrioritySort {
```

```
    public static void main (String [] args) {
```

```
        try (Scanner sc = new Scanner (System.in)) {
```

WebsiteResponseSort.java > WebsiteResponseSort > main(String[] args)

```
1 import java.util.Scanner;
2
3 public class WebsiteResponseSort {
4     Run main | Debug main
5     public static void main(String[] args) {
6         try( Scanner sc = new Scanner(System.in)){
7
8             System.out.println("Enter Number of Responses: ");
9             int n = sc.nextInt();
10
11             System.out.println("Enter Responses: ");
12             int[] response = new int[n];
13
14             for(int i = 0; i < n; i++) {
15                 response[i] = sc.nextInt();
16             }
17
18             for(int i = 1; i < n; i++) {
19
20                 int key = response[i];
21                 int j = i - 1;
22
23                 while(j >= 0 && response[j] > key) {
24                     response[j + 1] = response[j];
25                     j--;
26                 }
27
28                 response[j + 1] = key;
29             }
30
31             System.out.println("Responses in Sorted Order: ");
32             for(int i = 0; i < n; i++) {
33                 System.out.print(response[i] + " ");
34             }
35         }
36     }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Harshit Puri\DSA Java> cd "c:\Users\Harshit Puri\DSA Java\" ; if (\$?) { javac WebsiteResponseSort.java } ; if (\$?) { java WebsiteResponseSort }

Enter Number of Responses:

3

Enter Responses:

15

23

2

Responses in Sorted Order:

2 15 23

PS C:\Users\Harshit Puri\DSA Java> |

```
System.out.println("Enter no. of Reports: ");
```

```
int n = sc.nextInt();
```

```
System.out.println("Enter Reports: ");
```

```
int[] priority = new int[n];
```

```
for (int i=0; i<n; i++) {
```

```
    priority[i] = sc.nextInt();
```

```
}
```

```
for (int i=0; i<n-1; i++) {
```

```
    int minIndex = i;
```

```
    for (int j = i+1; j<n; j++) {
```

```
        if (priority[j] < priority[minIndex]) {
```

```
            minIndex = j;
```

```
        }
```

```
    }
```

```
    int temp = priority[i];
```

```
    priority[i] = priority[minIndex];
```

```
    priority[minIndex] = temp;
```

```
}
```

```
System.out.println("Reports in Sorted Order: ");
```

```
for (int i=0; i<n; i++) {
```

```
    System.out.print(priority[i] + " ");
```

```
}
```

```
}
```

```
}
```

```
}
```

Time Complexity : $O(N^2)$

Space Complexity : $O(1)$

BugReportPrioritySort.java > BugReportPrioritySort > main(String[] args)

```
1  import java.util.Scanner;
2
3  public class BugReportPrioritySort {
4      Run main | Debug main
5      public static void main(String[] args) {
6
7          try(Scanner sc = new Scanner(System.in)){
8
9              System.out.println("Enter Number of Reports: ");
10             int n = sc.nextInt();
11
12             System.out.println("Enter reports: ");
13             int[] priority = new int[n];
14
15             for(int i = 0; i < n; i++) {
16                 priority[i] = sc.nextInt();
17             }
18
19             for(int i = 0; i < n - 1; i++) {
20                 int minIndex = i;
21
22                 for(int j = i + 1; j < n; j++) {
23                     if(priority[j] < priority[minIndex]) {
24                         minIndex = j;
25                     }
26                 }
27
28                 int temp = priority[i];
29                 priority[i] = priority[minIndex];
30                 priority[minIndex] = temp;
31             }
32
33             System.out.println("Reports in Sorted Order: ");
34             for(int i = 0; i < n; i++) {
35                 System.out.print(priority[i] + " ");
36             }
37         }
38     }
39 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Harshit Puri\DSA Java> cd "c:\Users\Harshit Puri\DSA Java\" ; if ($?) { javac BugReportPrioritySort.java } ; if ($?) { java BugReportPrioritySort }
Enter Number of Reports:
3
Enter reports:
12
6
8
Reports in Sorted Order:
6 8 12
PS C:\Users\Harshit Puri\DSA Java> |
```